

DevOps and Analytics Transformation Strategy

Plain-language revised version

1. Purpose and recommended direction

The organization needs to bring work from **five Azure DevOps (ADO) organizations** into one **GitHub Enterprise** environment. The scenario includes about **1,000 repositories** and **1,500 pipelines**. Azure Boards is expected to remain in one ADO organization. The organization is still developing consistent DevOps practices, so the transformation must improve reliability and security without making it harder for teams to deliver software.

Recommendation: re-scope the work. Do not stop the entire program, and do not accelerate every migration just to meet a date. Keep moving repositories that are well understood and low risk. Put more attention on pipeline conversion, developer experience, and workloads that need special Windows or x86 build environments. Increase security requirements in stages, as the standard workflows prove stable. Treat disputed DORA data as a data-quality problem to solve—not as a way to grade teams or people.

This plan is a proposal based on the case-study scenario. It is not proof that the migration, GitHub setup, Fabric solution, or security controls have already been deployed.

2. What the future platform should look like

- **GitHub Enterprise** is the central place for source code, pull requests, and the standard CI/CD platform.
- **Azure Boards** remains the work-tracking system in one chosen ADO organization. The organization must test that links among Boards work items, GitHub branches, commits, and pull requests remain useful after migration.
- **Shared workflows** provide approved, reusable build, test, and security steps. Teams should not need to copy and maintain different versions of the same pipeline.
- **Hosted runners** serve standard builds. A separate, restricted runner pool serves approved legacy builds that need Windows, x86, native tools, or other special requirements.
- **Microsoft Fabric** provides the analytics platform. Bronze holds authorized source data as received, Silver cleans and standardizes it, and Gold contains reviewed, agreed metrics. Power BI can report from Gold once the data and metric definitions are trusted.
- A **service catalog** records each service's owner, criticality, repository, pipeline, support route, and relevant exceptions.

Central teams provide a safe and supported path. Product teams still own their software, tests, service information, deployment definitions, and incident facts.

3. Target operating model: who does what

Group	Main responsibilities
Executive sponsor	Sets priorities, resolves cross-organization issues, approves investment and risk decisions, and chairs major go/no-go reviews.
Platform product team	Builds and supports shared workflows, migration tools, runner services, documentation, developer support, and platform reliability.
Security and identity teams	Define access, identity, workflow, runner, audit, and exception requirements; help teams meet them safely.
Product and service teams	Maintain code and tests, name owners, describe release and deployment behavior, validate their migration, and confirm incident attribution.
Fabric and analytics team	Own data ingestion, data checks, metric definitions, lineage, access, freshness, and dashboard quality indicators.
Azure Boards owner	Maintains the selected ADO organization and confirms the Boards-to-GitHub links work as expected.

4. Treat the platform as an internal product

The platform is a product used by engineers. It needs a roadmap, a named product owner, technical owners, support, documentation, service expectations, and a way for teams to give feedback. Measure whether teams can get a first build working and receive useful feedback quickly—not just how many repositories have moved.

The recommended workflow should be the easiest supported choice. Teams can suggest improvements, but shared changes should be reviewed, tested, versioned, and explained before they are rolled out broadly. A small group of engineering champions can help test changes and bring feedback from different teams.

5. Security and governance in practical terms

Basic protections for every repository

- Use approved company identity and account-management controls.
- Give people and automation only the access they need. Review access when roles or ownership change.
- Protect important branches with reviews and required checks. Require specialist review for sensitive files where appropriate.
- Scan for exposed credentials and vulnerable software dependencies. Give teams a clear way to fix findings and ask for help.
- Keep workflow token permissions read-only unless a step has a justified need for more access. Do not store secrets in source code or print them in logs. Prefer short-lived federated credentials over long-lived tokens where the environment supports them.
- Use only approved workflow actions. Follow company policy for pinning external actions to fixed, reviewed versions.
- Protect production environments with the required approvals. Retain build logs, audit records, and artifacts for the time required by policy.
- Keep self-hosted runners separate from sensitive networks and from one another. Prefer disposable runners. Do not let untrusted pull-request code use a runner that contains sensitive access.

Roll out controls in a usable way

Security requirements should be agreed by Security and Engineering together. Apply the most important protections first. Before a check becomes mandatory everywhere, confirm that it runs reliably, gives understandable results, and has a team that can support it. If a check is noisy or unavailable, improve the check rather than encouraging teams to bypass it.

Track blocked pull requests and bypasses so the organization can find problems. These are signals for investigation, not simply numbers to use to blame developers.

Handle exceptions visibly

Some products cannot use the standard runner or workflow immediately. Record each exception with the affected repository or pipeline, the reason, the risk, compensating protections, accountable owner, approver, expiry date, and planned remediation. Review exceptions monthly. An exception should end when the workload can use the standard path or leadership renews it with a documented reason.

6. Migration plan: move in safe, understandable steps

First, create a useful inventory

For each repository and pipeline, record its owner, business criticality, language and runtime, build tools, dependencies, permissions, secrets, deployment destination, artifact consumers, test coverage, runner and architecture needs, release cadence, and rollback method. Identify shared templates and pipelines that produce artifacts used by other services. Confirm facts with owners; do not assume a pipeline deploys to production just because of its name.

Then migrate in waves

Wave	What happens	Exit condition
Wave 0: Discover and prepare	Confirm owners, inventory, risks, security baseline, Boards links, artifacts, and rollback plan. Choose representative pilots.	Pilot scope and success criteria are approved; the team knows how to restore the existing path.
Wave 1: Simple workloads	Move low-risk repositories with common tools and few dependencies. Run old and new validation together where useful.	Builds, tests, permissions, and outputs match expectations; owner accepts the result.

Wave	What happens	Exit condition
Wave 2: Common patterns	Convert groups of similar pipelines using shared workflows. Batch related repositories by service and dependency.	Shared pattern is stable, documented, supported, and accepted by service owners.
Wave 3: Special workloads	Address x86, native, regulated, or tightly coupled products using isolated runners or approved temporary exceptions.	Required runner, security, support, and exception controls are in place.
Wave 4: Retire old paths	Turn off old pipelines only after the new path is proven and rollback has been rehearsed.	Owners approve; artifacts, audit needs, access, and support are covered; retirement is recorded.

Do not migrate solely by repository count. Group work so service dependencies and release responsibilities are understood. Azure Boards remains in scope for integration validation, not for a forced platform migration unless executives make that a separate decision.

7. How to validate a migration

A workflow file existing in a repository is not enough to prove that a migration is successful. For each pilot or migration batch:

- **Review the workflow:** check syntax, permissions, action versions, secrets, runner type, and file paths.
- **Compare the build:** confirm the new pipeline uses the required operating system, architecture, toolchain, configuration, and dependencies.
- **Run the tests:** compare test results with the old pipeline and publish coverage in a consistent format. A successful compile alone is not sufficient.
- **Compare artifacts:** verify names, contents, retention, signing needs, and downstream consumers.
- **Check security:** verify access, branch rules, secret and dependency checks, approvals, and runner isolation.
- **Test failure handling:** simulate a failed build or runner and confirm the team can find logs, understand the failure, and recover.
- **Get service-owner approval:** the owner confirms that the new pipeline is acceptable before the old path is retired.

Keep evidence with the migration record: workflow run, build and test results, coverage, artifact information, approvals, known differences, exception approval if applicable, and rollback rehearsal. A green pull-request check does not by itself prove that a production release is safe.

The supplied Azure pipeline uses a Windows runner, x86 build settings, the v143 toolset, a Visual C++ x86 redistributable, NuGet restore, and VSTest coverage. Do not change it to Linux or x64 without checking the actual project requirements and getting the owners' approval.

8. Rollback: how to return safely to the previous path

Rollback means being able to restore a known-good build or release process without losing source history, artifacts, or audit evidence.

- Keep the previous pipeline available during a defined dual-run period. Make sure only one pipeline can deploy to production at a time.
- Preserve repository history and required release artifacts according to retention rules.
- Document a clear switch or procedure for returning to the old pipeline if the new one blocks a critical release, creates incorrect output, or fails an important control.
- Agree on rollback triggers before cutover. Examples include repeated unexplained failures, missing artifacts, failed security protections, inability to restore service, or unacceptable developer impact.
- Name who may decide to roll back, who performs the change, and who communicates it.
- Save relevant logs and evidence, investigate the cause, and repeat the validation before trying the cutover again.
- Set an end date and exit conditions for dual running so duplicate systems do not remain forever.

If a gate fails, pause the affected workload or migration wave. Do not automatically stop every migration unless the issue is broad enough to justify it.

9. Trusted analytics and DORA metrics

Before publishing targets, agree what each metric means. Document its service and environment, time zone, source events, denominator, duplicate handling, exclusions, attribution rules, data owner, freshness expectation, and how missing or poor-quality data will be shown.

Metric	Proposed meaning	Important requirement or limitation
Deployment Frequency	Count of successful production deployments for a service during a stated period.	A build is not automatically a production deployment. Use deployment records with environment and outcome.
Lead Time for Changes	Time from an agreed change event, such as the first commit included in a release, until successful production deployment.	Requires trustworthy links from deployed releases to included commits. State how timestamps are chosen.
Change Failure Rate	Qualifying production deployments that cause an agreed incident, divided by eligible production deployments.	Requires an agreed incident definition, attribution window, deployment denominator, and deduplication rule.
Mean Time to Restore	Average time from customer-impact start until service is restored for qualifying incidents.	Normalize time zones, reject invalid intervals, and show event counts and unresolved incidents as context.

The workbook was profiled and includes 164 commits, 350 build records, 11 incidents, and two projects. However, the commits sheet uses `build_sk` while the builds sheet uses `build_id`; the relationship needs validation. The build records do not contain an obvious success/failure result or a clear production-deployment event. Only three incidents are marked deployment-caused, and one incident has a resolution time earlier than its start. Therefore, the workbook alone does not support a complete, defensible DORA baseline. Confirm the source meaning with system owners, quarantine invalid records, and show data coverage and quality alongside any measure. Do not rank teams or individuals using incomplete data.

10. AI Ops and Agentic DevOps opportunities

Start with low-risk assistance using only approved enterprise tools and approved information:

- Answer platform questions using reviewed documentation.
- Summarize common causes of failed builds and suggest relevant troubleshooting steps.
- Draft a possible Azure-to-GitHub workflow conversion for a human to review.
- Highlight unusual changes in pipeline reliability or metric quality for an engineer to investigate.

Every generated code or configuration change should go through normal tests, security checks, and human review. Do not initially allow an AI agent to merge code or deploy to production on its own. Do not provide secrets or restricted data to unapproved AI services. Measure whether the assistance saves time and remains accurate; stop pilots that create more rework or risk than value.

11. Risks and responses

Risk	Plain-language response
Pipeline conversion falls further behind	Focus on common pipeline types, assign owners, and use small waves with visible progress and support.
Legacy x86 or native builds do not work on standard runners	Provide a restricted, supported Windows/x86 runner path and time-bound modernization exceptions.
Security rules add friction or encourage workarounds	Roll out in stages, improve noisy checks, provide support, and audit exceptions and bypasses.
Migration breaks a release or loses artifacts	Dual-run where appropriate, compare artifacts, preserve the old route temporarily, and rehearse rollback.
DORA numbers are disputed or misleading	Agree definitions, verify event links, track data quality, show gaps, and avoid ranking teams.
AI exposes information or suggests unsafe changes	Use approved tools, limit access, require human review, audit use, and prohibit autonomous production changes at the start.

Risk	Plain-language response
Central platform team becomes a bottleneck	Treat the platform as a product, publish supported patterns, involve team champions, and provide a clear contribution route.

12. Twelve-month roadmap

Period	Main work	Expected result
Months 1–3	Inventory owners and pipeline types; define risk tiers and security baseline; pilot standard and exception workflows; agree metric definitions and identify data gaps; measure developer friction.	Clear ownership, representative pilot evidence, known special runner needs, and an agreed plan for data quality.
Months 4–6	Expand reusable workflows and migration support; establish isolated runner capability; test GitHub–Boards links; build the first Fabric Bronze/Silver/Gold flow and quality reporting.	Repeatable migration patterns and an initial, clearly qualified analytics product.
Months 7–9	Migrate service-aligned waves; retire old paths only after gates pass; expand lineage, metric coverage, and graduated security enforcement.	More services on supported paths, with evidence and fewer unmanaged exceptions.
Months 10–12	Resolve remaining exceptions with owners; optimize reliability and cost; audit access, artifacts, controls, and metric quality; set the next-year roadmap.	A sustainable platform and a prioritized next phase based on measured outcomes.

13. Prioritized first 90 days

Days 0–30: understand and agree

- Name the executive sponsor, platform product owner, service owners, security contacts, and analytics steward.
- Create a consistent inventory of repositories, pipelines, owners, risks, dependencies, and runner needs.
- Identify Windows/x86 and native dependencies and confirm what must remain supported.
- Record existing build outputs, release behavior, security controls, and rollback methods.
- Agree pilot selection, security minimums, exception fields, migration measures, and metric definitions.
- Speak with developers about the most painful parts of the current build and release process.

Days 31–60: prove the approach

- Pilot roughly 20–30 representative repositories, including standard and exception workloads.
- Build and review reusable workflow patterns for common pipeline types.
- Compare old and new build, test, coverage, and artifact results.
- Dual-run critical paths when that reduces risk; prevent two systems from deploying simultaneously.
- Add data-quality checks and document the DORA source gaps.
- Run regular help sessions and use feedback to improve the templates.
- Review runner isolation and workflow permissions with Security.

Days 61–90: scale only what passes

- Expand the low-risk wave only if readiness checks pass.
- Apply stronger branch protections to workflows that are stable and supported.
- Resolve or formally time-limit the highest-priority runner exceptions.
- Present provisional analytics with definitions, lineage, coverage, and data-quality warnings.
- Hold a leadership gate to decide the next wave based on delivery speed, reliability, rollback readiness, security, developer friction, cost, and analytics quality.

14. Investment, measures, and work to defer

Prioritize investment in a small platform product team, migration automation, developer support, secure Windows/x86 runner capacity, identity and security engineering, and Fabric/data stewardship. Accept some temporary cost for dual running when it materially lowers cutover risk, but define when that cost ends.

Track repository and pipeline owner coverage, migration throughput, build/test success, time to useful feedback, rollback frequency, shared workflow adoption, runner availability and cost, critical security findings, age of exceptions, developer feedback, and DORA source coverage and freshness.

Defer mass rewrites, forced Azure Boards migration, a large self-hosted runner fleet for every workload, blanket mandatory controls for unsupported workflows, team-level DORA targets, and autonomous AI agents. Revisit these choices when evidence, product ownership, and reliable support are in place.

15. Executive decisions needed

- Confirm the sponsor, accountable owners, business priorities, and service-aligned migration order.
- Approve the minimum security baseline, staged rollout, exception process, and runner-isolation investment.
- Confirm Azure Boards remains in one ADO organization and identify who owns the integration.
- Approve Fabric data access, data stewardship, metric definitions, and a policy against ranking teams until data quality is acceptable.
- Agree the go/no-go gates for parity, reliability, security, artifact handling, rollback, and developer experience.

Assumptions and implementation boundary

The scale figures and destination direction come from the candidate brief. This document is a proposed strategy. It does not claim that an enterprise migration has taken place. The sample workflow and code require independent review and execution in the candidate's environment. The Fabric notebook is a starting point, not a deployed workspace. Do not publish the supplied workbook or its personal identifiers. Keep synthetic sample data clearly labeled and use approved, sanitized data only when authorized.